

Trabalho 1 (T1) – STC002

Prof. Laércio Pioli

< Algoritmo de ordenação: Odd-Even Sort >



Discentes: Caio Lene, Gabriel
Phillipy, João Pedro Sabino, Matheus
Dourado e Renan Moraes



O que é?

- **Definição:** algoritmo de ordenação baseado em comparações entre pares de elementos adjacentes, que possui semelhança com o Bubble Sort, mas que possui duas fases: par e ímpar.
- **Fase Par:** os elementos das posições $(0,1), (2,3), \dots$ são comparados.
- **Fase Ímpar:** os elementos das posições $(1,2), (3,4), \dots$ são comparados.
- **Execução das Fases:** essas duas fases são repetidas até que nenhuma troca seja feita durante uma passagem completa no vetor, indicando que o vetor está ordenado.

Como Funciona?

➤ **Vetor:** [4, 2, 3, 1]

➤ 1ª Passagem - Fase Par:

- Compara os elementos partindo dos índices pares (0,1) e (2,3).

$$\begin{array}{cccc} 4 & 2 & 3 & 1 \\ \wedge & _ & _ & \wedge \\ \wedge & _ & _ & \wedge \end{array} \rightarrow [2, 4, 1, 3]$$

➤ 1ª Passagem - Fase Ímpar:

- Compara os elementos partindo dos índices ímpares (1,2).

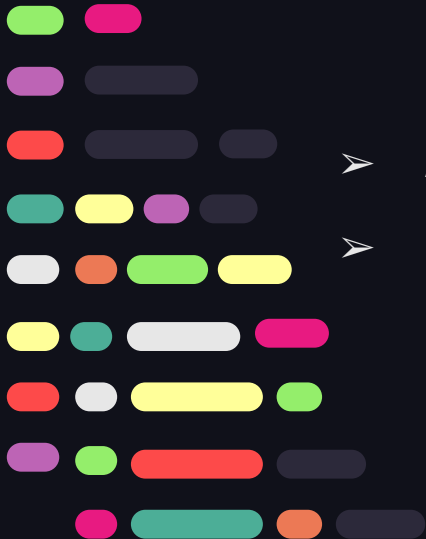
$$\begin{array}{cccc} 2 & 4 & 1 & 3 \\ \wedge & _ & _ & \wedge \\ \wedge & _ & _ & \wedge \end{array} \rightarrow [2, 1, 4, 3]$$

➤ As demais passagens seguem a mesma lógica até que não aconteça nenhuma troca durante uma passagem completa no vetor e o loop se encerre!



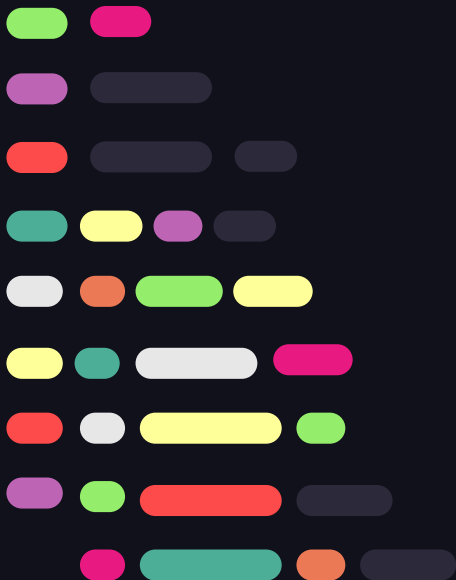
Paralelismo

- Dentro de cada fase, par e ímpar, todas as comparações são independentes entre si, ou seja, o par (0,1) não interfere no par (2,3), e assim por diante.
- Em sistemas com múltiplos processadores, isso permite que todos os pares de uma fase sejam comparados simultaneamente.
- Aplicação prática: computação paralela.
- Nossa implementação em C é sequencial, mas a estrutura do algoritmo permite essa execução paralela, que por sua vez resulta em uma menor complexidade.



Pseudocódigo

```
1  procedimento oddEvenSort(A: tabela, N: inteiro)
2      troca = Verdadeiro
3      enquanto (troca = Verdadeiro) faça
4          troca = Falso
5          para i=0 até N-2 passo 2 faça
6              se A[i] > A[i+1] então
7                  aux ← A[i]
8                  A[i] ← A[i+1]
9                  A[i+1] ← aux
10                 troca ← Verdadeiro
11             fim-se
12         fim-para
13         para j=1 até N-2 passo 2 faça
14             se A[j] > A[j+1] então
15                 aux ← A[j]
16                 A[j] ← A[j+1]
17                 A[j+1] ← aux
18                 troca ← Verdadeiro
19             fim-se
20         fim-para
21     fim-enquanto
22 fim-procedimento
```



Implementação em C

```
1  #include <stdio.h>
2
3  void OddEvenSort(int A[], int tam);
4
5  int main() {
6      int V[] = {9, 2, 5, 1, 7, 13, -2, -1, 0};
7      int TAM = sizeof(V) / sizeof(V[0]);
8      OddEvenSort(V, TAM);
9      return 0;
10 }
```

```
12 void OddEvenSort(int A[], int tam){
13     int troca = 1;
14     while (troca) {
15         troca = 0;
16         for (int i = 0; i <= tam - 2; i += 2){
17             if(A[i] > A[i+1]){
18                 int aux = A[i];
19                 A[i] = A[i+1];
20                 A[i+1] = aux;
21                 troca = 1;
22             }
23         }
24         for(int j = 1; j <= tam - 2; j += 2){
25             if(A[j] > A[j+1]){
26                 int aux = A[j];
27                 A[j] = A[j+1];
28                 A[j+1] = aux;
29                 troca = 1;
30             }
31         }
32     }
33 }
```

Análise de Complexidade

```
12 void OddEvenSort(int A[], int tam){
13     int troca = 1;
14     while (troca) {
15         troca = 0;
16         for (int i = 0; i <= tam - 2; i += 2){
17             if(A[i] > A[i+1]){
18                 int aux = A[i];
19                 A[i] = A[i+1];
20                 A[i+1] = aux;
21                 troca = 1;
22             }
23         }
24         for(int j = 1; j <= tam - 2; j += 2){
25             if(A[j] > A[j+1]){
26                 int aux = A[j];
27                 A[j] = A[j+1];
28                 A[j+1] = aux;
29                 troca = 1;
30             }
31         }
32     }
33 }
```

Analisando as Estruturas de Repetição

Estrutura	Linha	Número Vezes
Loop externo	14	n
1º loop interno	16	n/2
2º loop interno	24	n/2

Na notação Big-0, ignora-se as constantes:

Dois for sequenciais: $O(n) + O(n) = O(n)$

For aninhado ao while: $O(n) * O(n) = O(n^2)$



Tabela de Complexidade

{

Caso	Complexidade
Melhor	$O(n)$
Médio	$O(n^2)$
Pior	$O(n^2)$

}



Comparação de Complexidades

{

Algoritmo	Melhor	Médio	Pior
Odd-Even Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

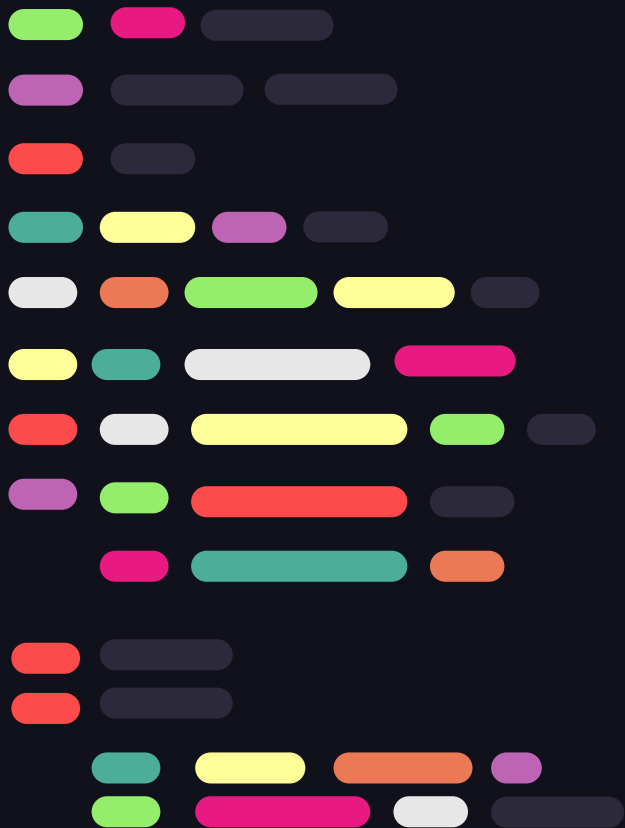
}

Código em Execução

{

Apresentação do código em funcionamento!

}



Fim! }

< Agradecemos a atenção de todos! >

